Swinburne University of Technology
School of Science, Computing, and Emerging Technologies

SWE40006 - Software Deployment and Evolution

Deployment Portfolio Task 3

Student name: Ngoc Van Nhi Do
Student ID: 105551765
Submission date: DON'T FORGET THISSSSSSSSSSS

Deployment Portfolio Task 3

1 Declared Target Level

All four subtasks 3.1, 3.2, 3.3, and 3.4 were attempted.

Table 1 below contain relevant links as proof of successful task execution.

Table 1: Publicly accessible URLs for live grading verification

Source	URL
Task 3.1	http://13.55.158.77

2 Execution Evidence

2.1 Task 3.1

For this task, I deployed a WordPress application onto an Amazon EC2 instance.

2.1.1 AWS Environment Setup

First, I created a new AWS account on the free plan, which gave me $100 in free credits.

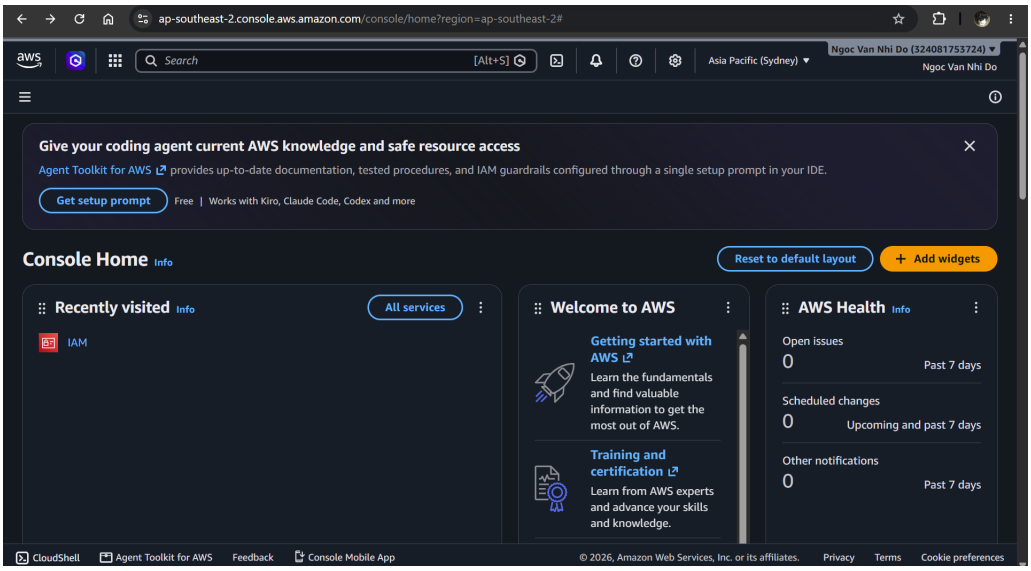

Figure 1: The AWS Console interface after I had successfully signed up.

I then created a new IAM user for my coursework named `SWE40006_student` rather than using the root account. This user is assigned to a new user group `EC2-Lab-Users` which is given full access to EC2 resources (using `AmazonEC2FullAccess`).

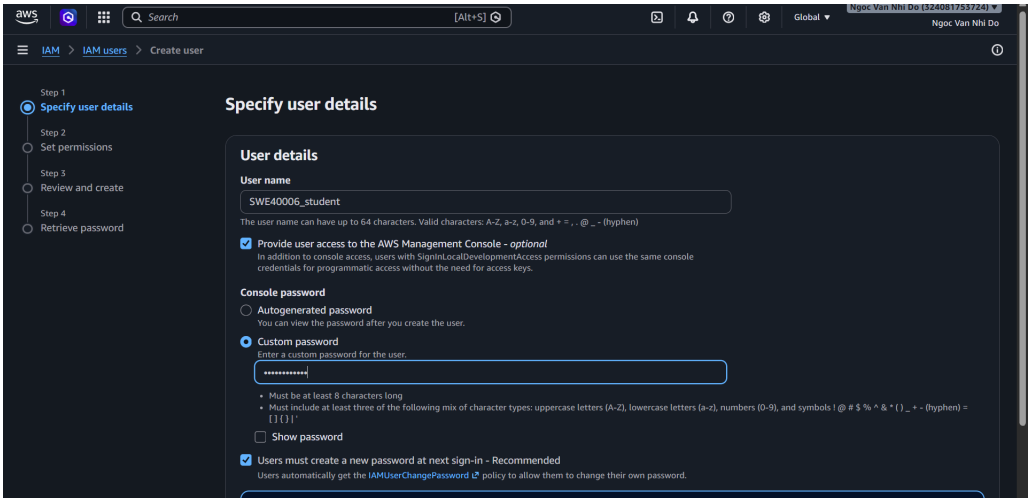

Figure 2: Creating a new IAM user `SWE40006_student`.

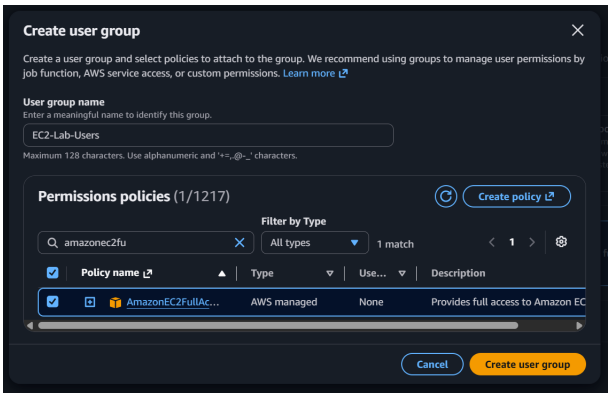

Figure 3: Adding `AmazonEC2FullAccess` to the user group `EC2-Lab-Users`, which is then assigned to `SWE40006_student`.

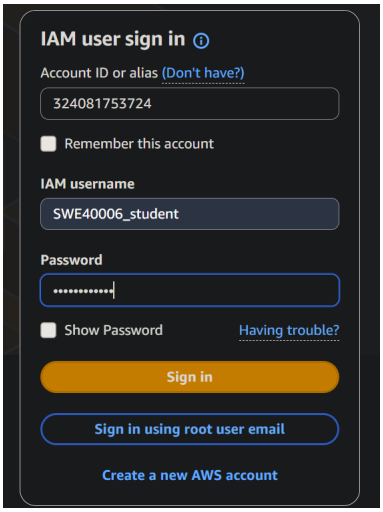

Figure 4: Logging into the AWS Console as the new IAM user.

To securely login into my EC2 instances, I must generate an SSH key pair. The private key was downloaded and stored on my local machine for later authentication.

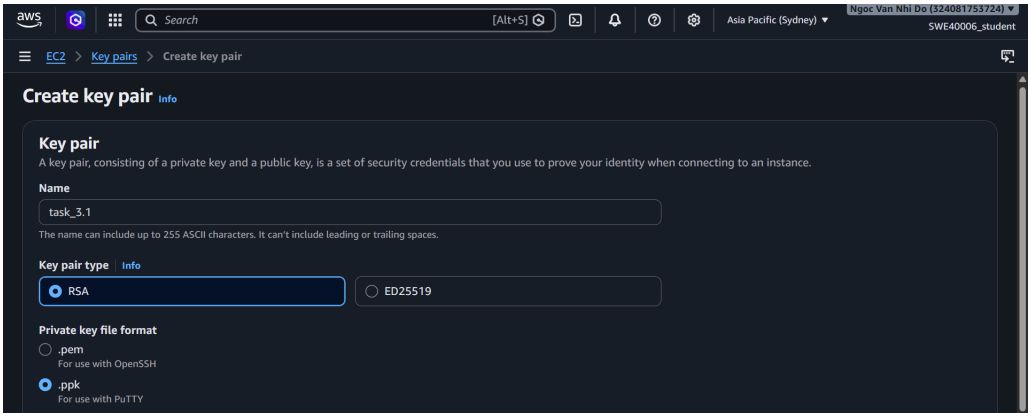

Figure 5: Creating a new key pair `task-3.1`.

2.1.2 Compute Provisioning

I launched a new EC2 instance called `Task 3.1`. This instance runs Amazon Linux 2023, uses a `t3.micro` instance time to remain eligible for the free tier, and is configured with the previously generated key pair.

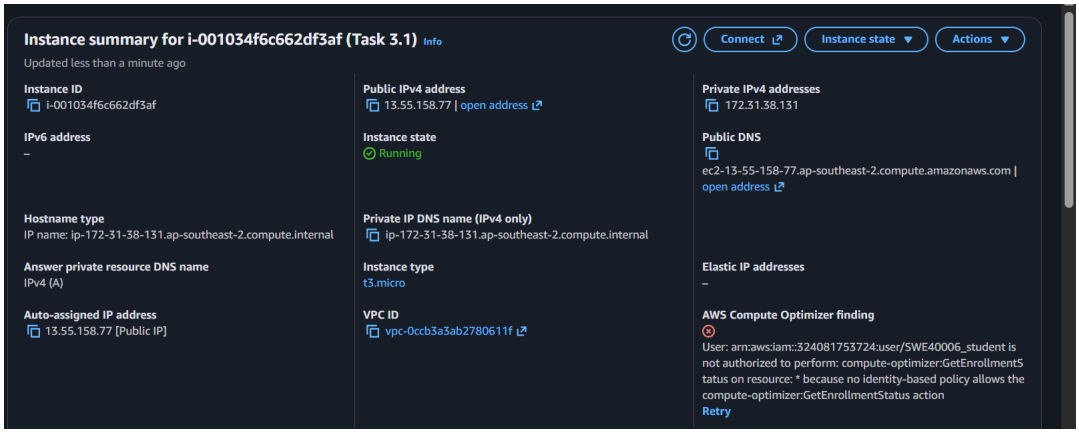

Figure 6: Creating a new Amazon EC2 instance.

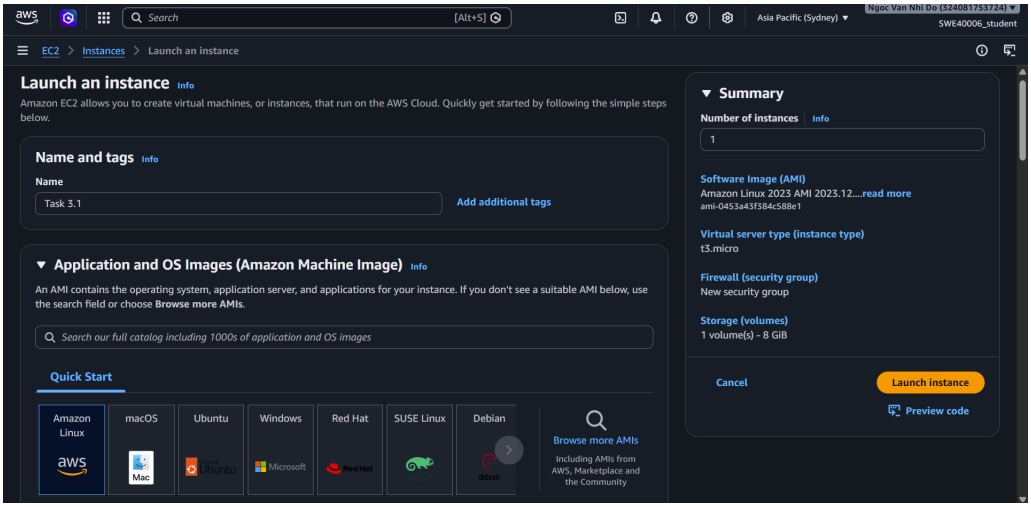

Figure 7: Amazon Linux 2023 was chosen as the OS for this instance.

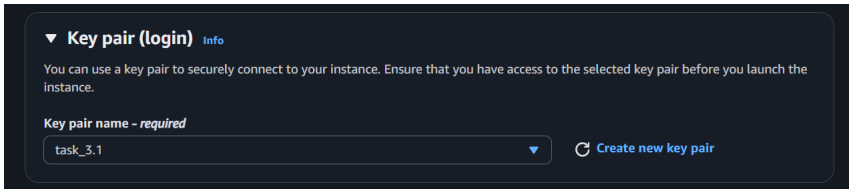

Figure 8: Configuring the instance with the `task-3.1` key pair.

I then configured the instance with a new security group `task-3.1-sg`, which allowed SSH (22), HTTP (80), and HTTPS (443) inbound traffic from anywhere.

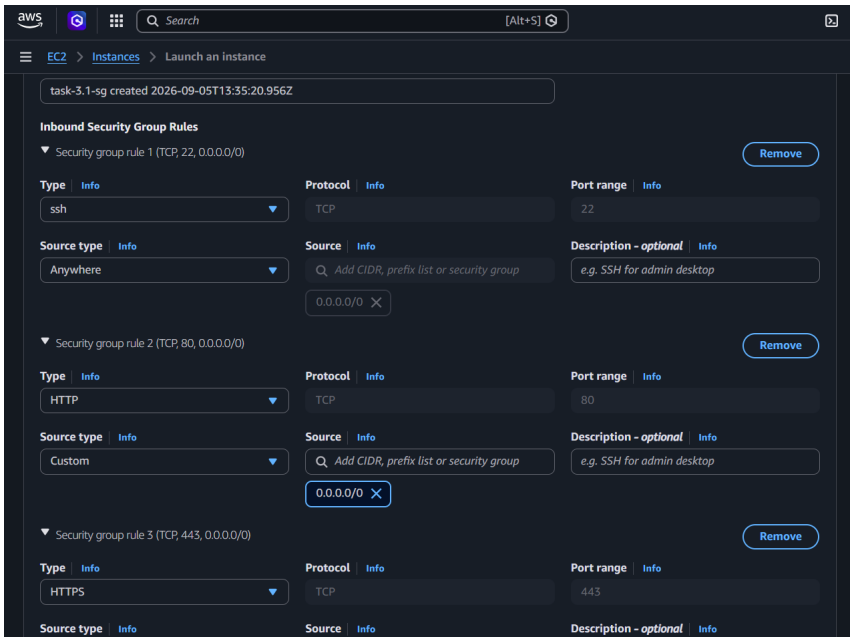

Figure 9: Configuring the security rules to allow inbound SSH, HTTP, and HTTPS traffic to this instance.

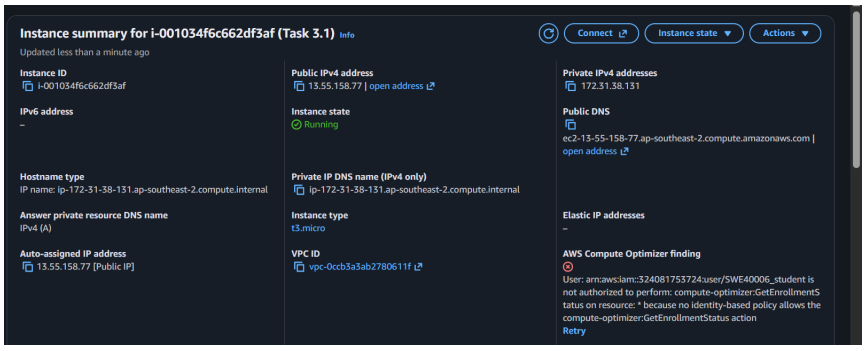

Figure 10: The new instance is successfully created.

I then SSH-ed to the instance using PuTTY.

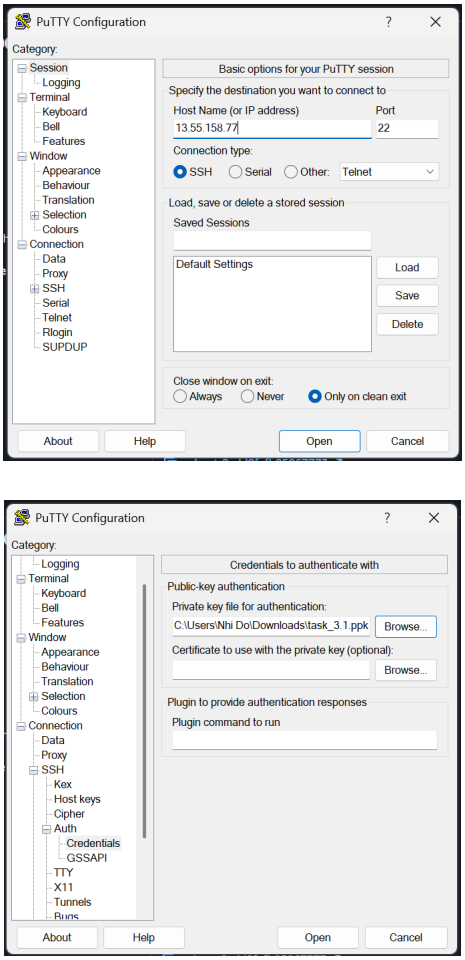

Figure 11: Connecting to the EC2 instance using SSH.

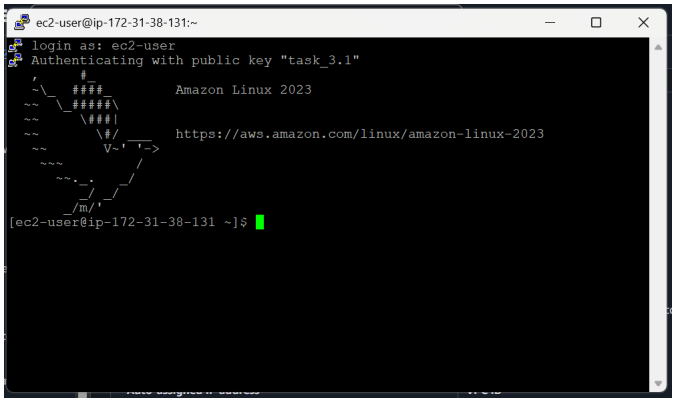

Figure 12: Connection was successful.

I then installed the Apache web server and started it so that it could later host my web application.

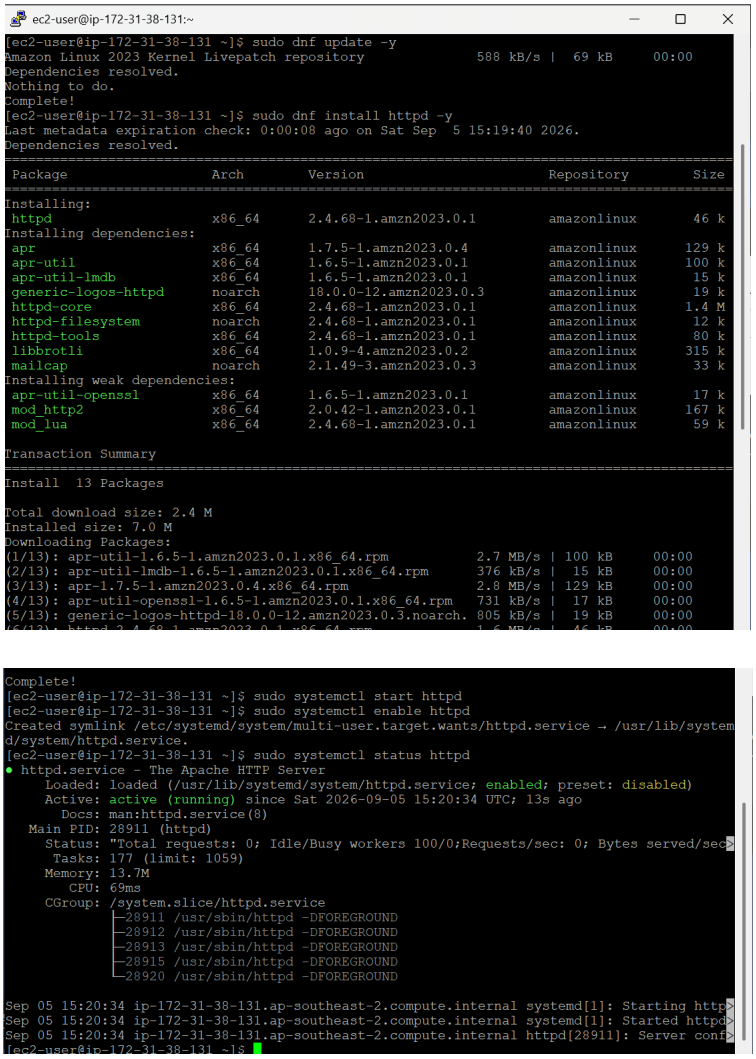

Figure 13: Installing and starting the web server.

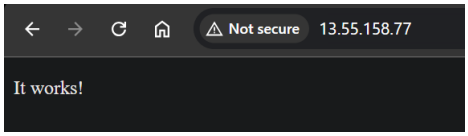

Figure 14: Verifying that the web server is publicly accessible.

2.1.3 Application Deployment

Before deploying the WordPress application, I needed to install its dependencies which included installing PHP and MariaDB.

Figure 15: Installing PHP.

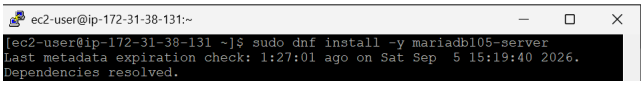

Figure 16: Installing MariaDB.

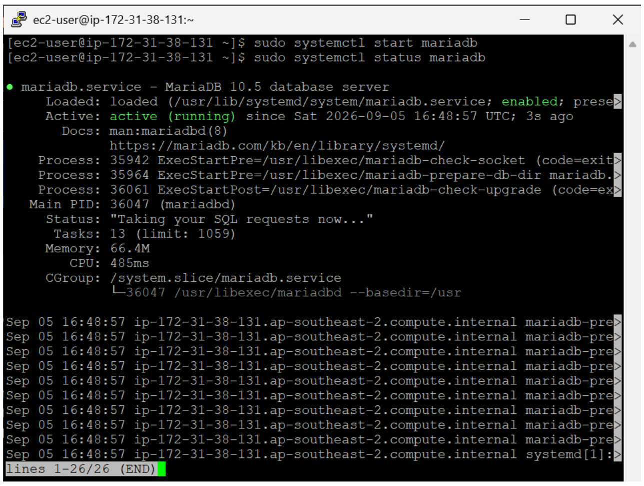

Figure 17: Starting and enabling the MariaDB service.

I then set up a local database for the web application.

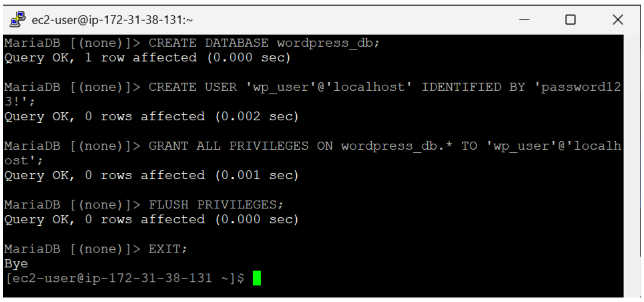

Figure 18: Setting up a local database named `wordpress_db` for the WordPress application.

I installed and extracted the WordPress package. The command `chown` made Apache the owner of the WordPress files, while `chmod` set who could read, write, or access those files. These commands ensured that Apache had the correct permissions to run WordPress.

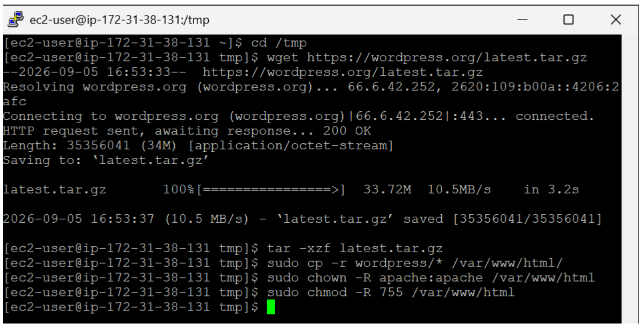

Figure 19: Installing WordPress and configuring permissions.

My WordPress application is deployed and running at http://13.55.158.77.

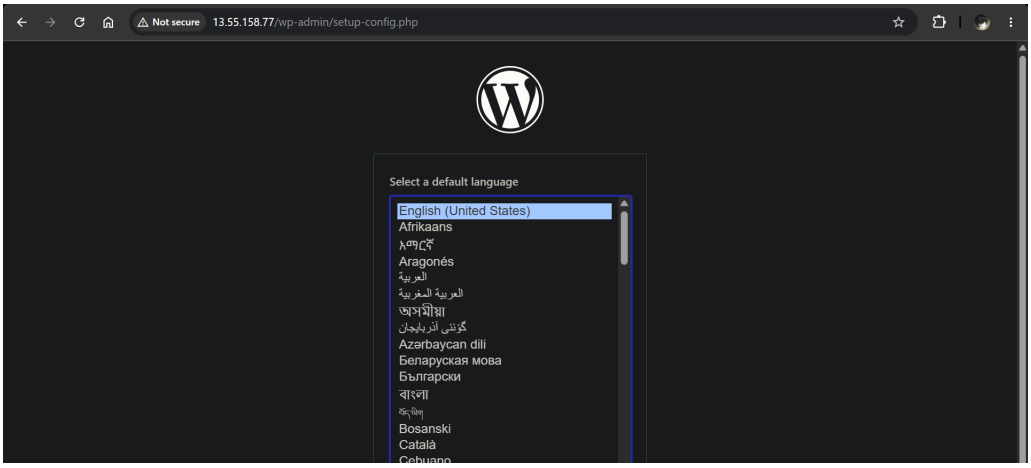

Figure 20: The deployed WordPress application.

2.2 Task 3.2

For this task, I configured my web application to use a database hosted on an Amazon RDS instance, effectively decoupling the database from the EC2 instance. I also created a backup of my web application by storing my files on an S3 bucket, and demonstrated how I could restore my application using this backup.

2.2.1 Database Decoupling

I started by creating a new RDS instance and configured it to use MariaDB as the database engine. I also connected the EC2 instance to this database right away, so that AWS automatically allowed inbound traffic on port 3306 from the web application to the database for me.

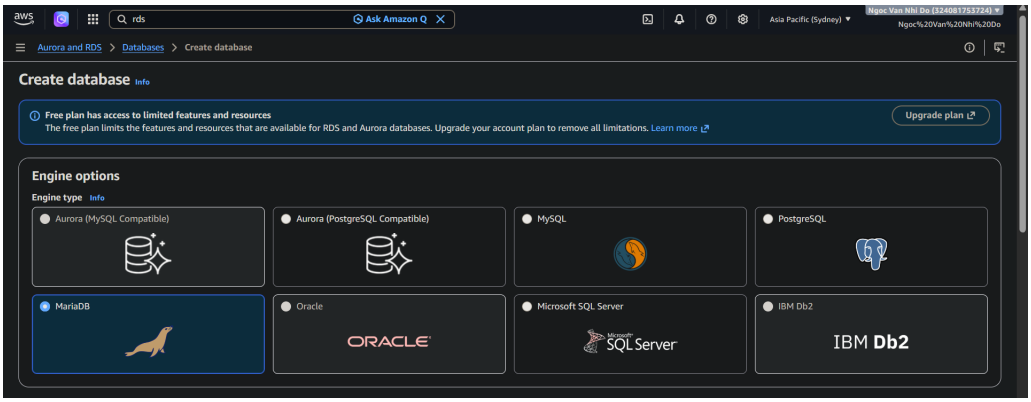

Figure 21: Creating a new RDS instance.

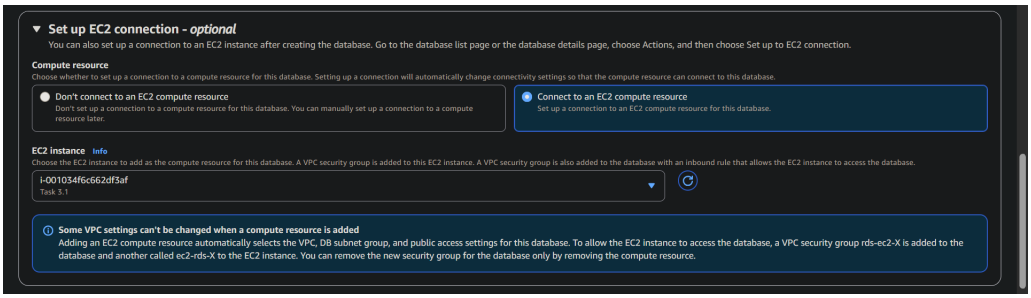

Figure 22: Connecting the EC2 instance from the previous task.

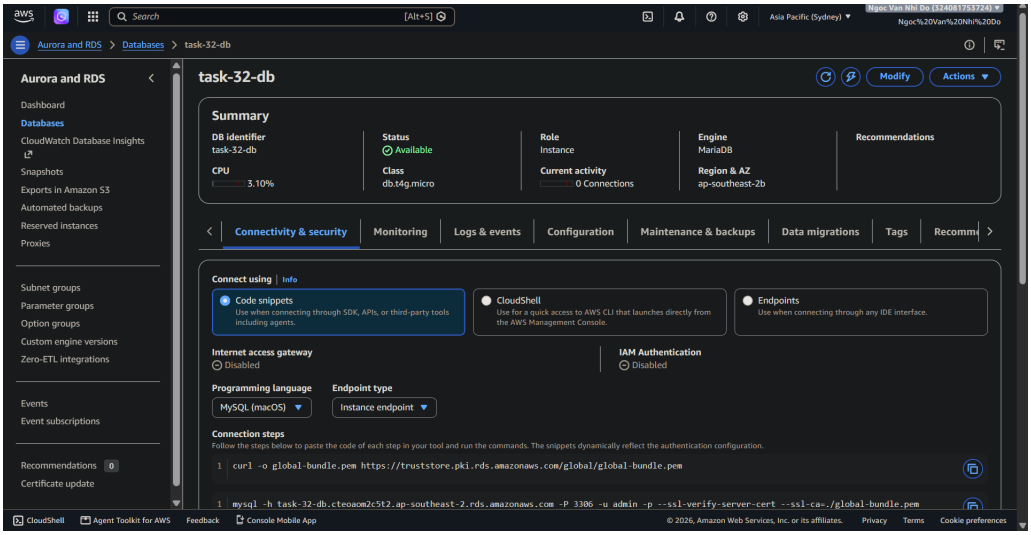

Figure 23: The database is successfully created.

From the EC2 instance's command line, I established a connection to the RDS instance using its endpoint at task-32-db.cteoaom2c5t2.ap-southeast-2.rds.amazonaws.com.

Figure 24: Connecting to the database from the EC2 instance's terminal.

Once I could connect to the database, I created a new database `wordpress_db` to be used by my application. I then exited the DB connection, configured the [wg_config.php] file with the required database information, including the database name, username, and password, along with the other necessary credentials. This allowed the WordPress application to connect to and use the configured database. Finally, I restarted the Apache web server and verified that the application has been successfully configured to use the database.

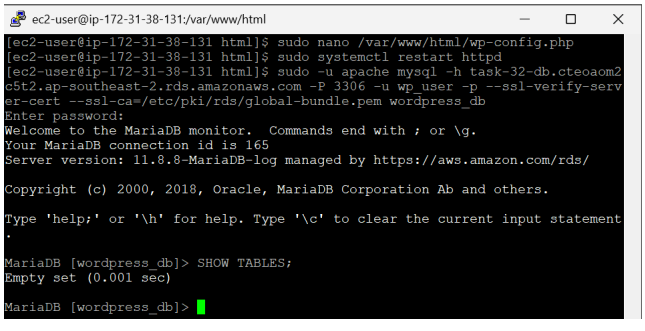

Figure 25: Editing the database configurations and verifiying the connection to the RDS database.

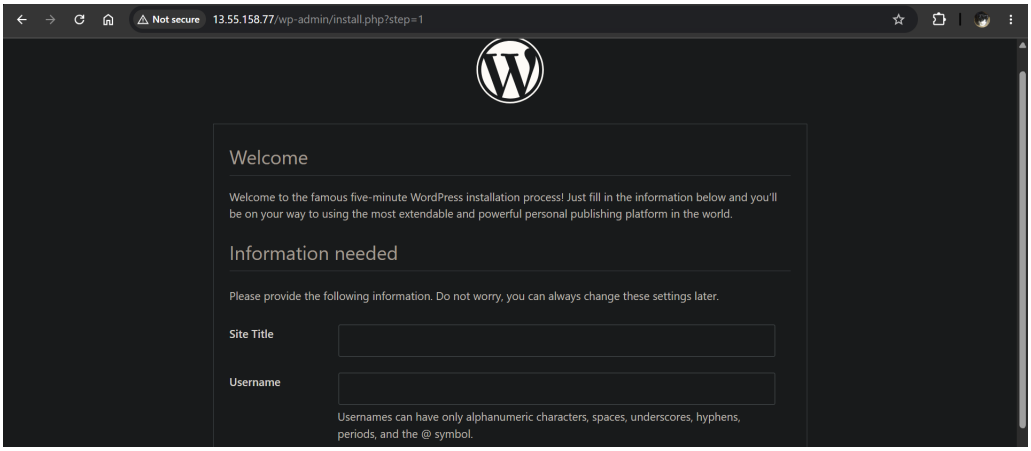

Figure 26: The application now shows the WordPress installation form, which means the database has successfully been connected.

2.2.2 Backup and Restore

I started by creating a new S3 bucket with the name `swe40006-105551765-bucket` to store my backup files.

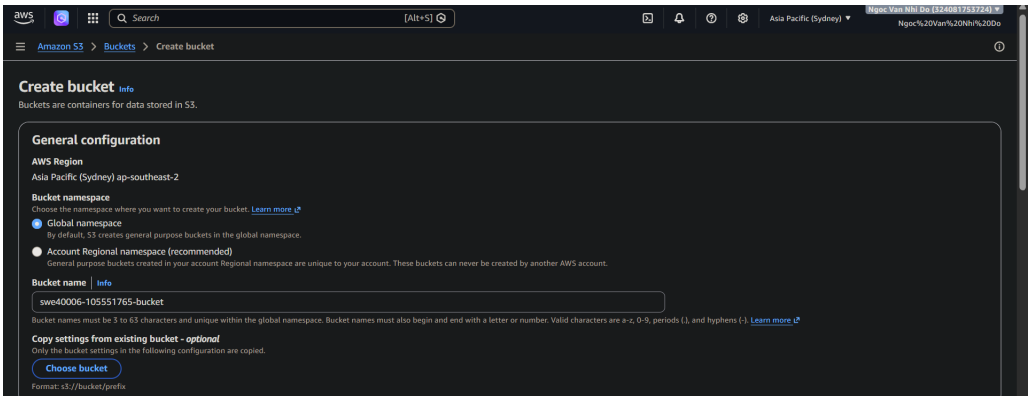

Figure 27: The application now shows the WordPress installation form, which means the database has successfully been connected.

I also created a new IAM role allowing full access to S3. This role is then attached to my EC2 instance so that the server could read and write files to the S3 bucket.

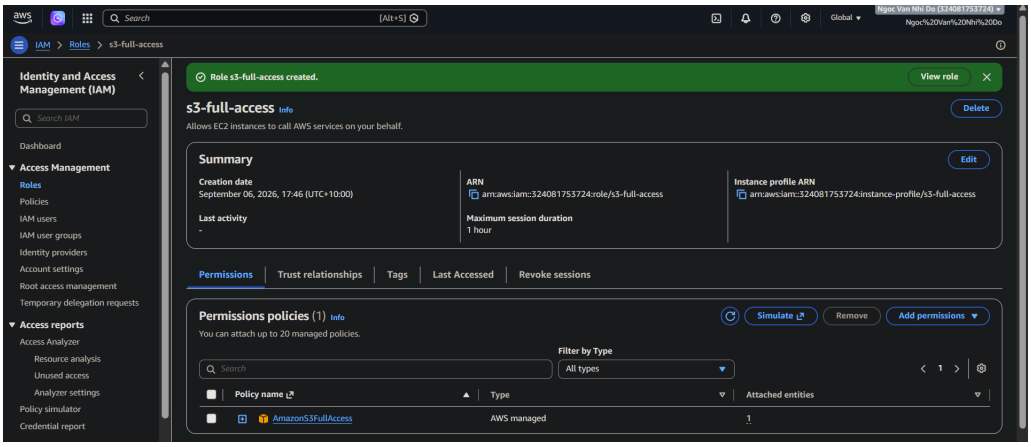

Figure 28: Creating a new IAM role with the `AmazonS3FullAccess` policy.

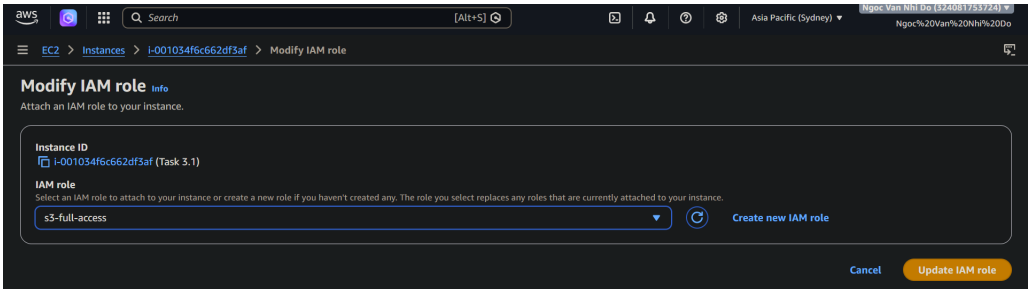

Figure 29: Adding the role to my EC2 instance.

I then made a backup of my files by archiving them into a `.tar` file to avoid having to individually handle hundreds of different files. This archive is then uploaded to the S3 bucket, serving as the backup to my web server.

```
html/wp-includes/widgets/class-wp-widget-tag-cloud.php
html/wp-includes/widgets/class-wp-widget-text.php
html/wp-includes/widgets.php
html/wp-includes/wp-db.php
html/wp-includes/wp-diff.php
html/wp-links-opml.php
html/wp-load.php
html/wp-login.php
html/wp-mail.php
html/wp-settings.php
html/wp-signup.php
html/wp-trackback.php
html/xmlrpc.php
html/wp-config.php
html/.htaccess
[ec2-user@ip-172-31-38-131 ~]$ ls -lh wordpress-backup.tar.gz
-rw-r--r--. 1 root root 34M Sep  6 06:26 wordpress-backup.tar.gz
[ec2-user@ip-172-31-38-131 ~]$
```

Figure 30: Creating the backup as a `.tar` archive.

```
ec2-user@ip-172-31-38-131:~
[ec2-user@ip-172-31-38-131 ~]$ aws s3 cp wordpress-backup.tar.gz s3://swe40006-105551765-bucket
upload: ./wordpress-backup.tar.gz to s3://swe40006-105551765-bucket/wordpress-backup.tar.gz
[ec2-user@ip-172-31-38-131 ~]$
```

Figure 31: Uploading the backup to the S3 bucket.

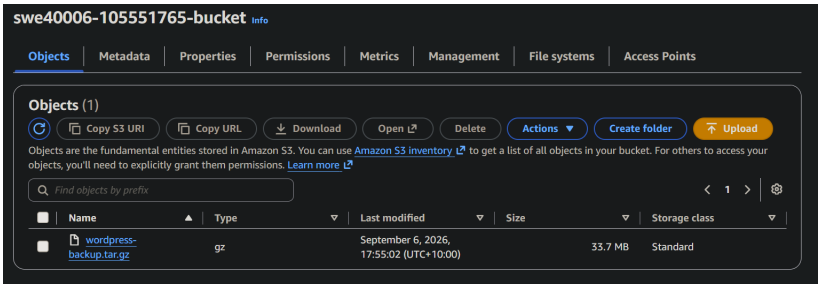

Figure 32: I could see that the backup has been uploaded using the AWS Console.

To demonstrate that I can restore my web server, I first removed all my web application files to simulate a deletion.

```
ec2-user@ip-172-31-38-131:~
[ec2-user@ip-172-31-38-131 ~]$ aws s3 cp s3://swe40006-105551765-bucket/wordp
ress-backup.tar.gz .
download: s3://swe40006-105551765-bucket/wordpress-backup.tar.gz to ./wordpre
ss-backup.tar.gz
[ec2-user@ip-172-31-38-131 ~]$ ls -lh ~/wordpress-backup.tar.gz
-rw-r--r--. 1 ec2-user ec2-user 34M Sep  6 07:55 /home/ec2-user/wordpress-bac
kup.tar.gz
[ec2-user@ip-172-31-38-131 ~]$ sudo rm -rf /var/www/html/*
[ec2-user@ip-172-31-38-131 ~]$ sudo rm -rf /var/www/html/.[!.]*
[ec2-user@ip-172-31-38-131 ~]$ ls -la /var/www/html/

total 0
drwxr-xr-x. 2 apache apache  6 Sep  6 07:59 .
drwxr-xr-x. 4 root   root   33 Sep  5 15:19 ..
[ec2-user@ip-172-31-38-131 ~]$
```

Figure 33: Removing all web application files on my EC2 instance.

Once the files were removed, accessing my website displayed only an "It works!" message rather than

the WordPress application, which is the default Apache web server page. This confirmed that the web application had been completely deleted.

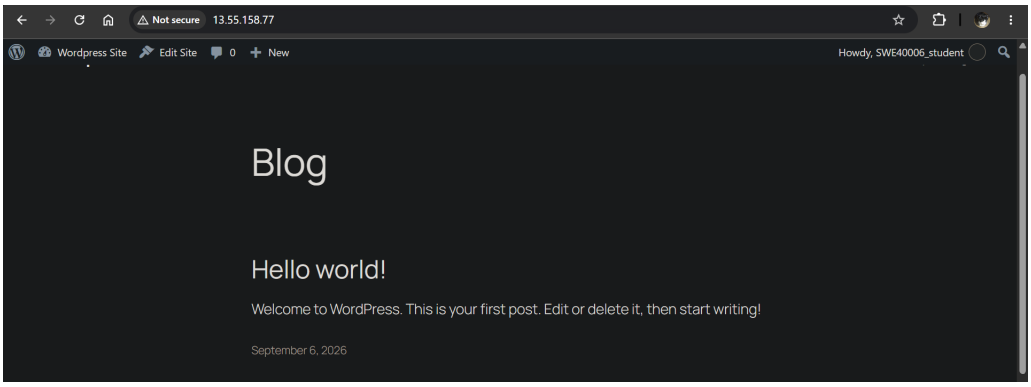

Figure 34: The WordPress application had been deleted.

To restore my deleted web application, I pulled the backup from the S3 bucket and extracted the archived files. The final step was to modify Apache's permissions to be able to read these files and restarting the web server.

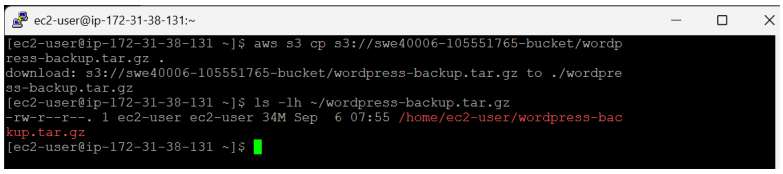

Figure 35: Downloading the backup from the S3 bucket.

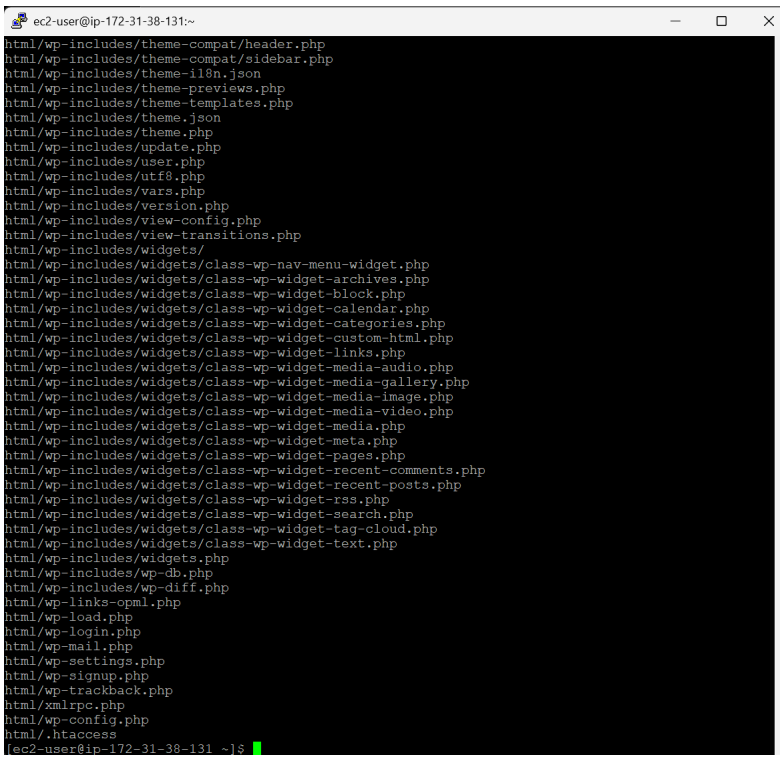

Figure 36: Extracting the web application files.

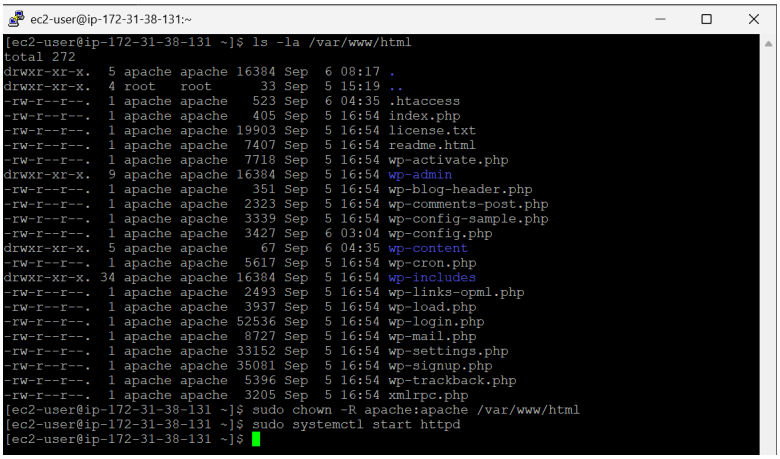

Figure 37: Verifying that the application files have been extracted.

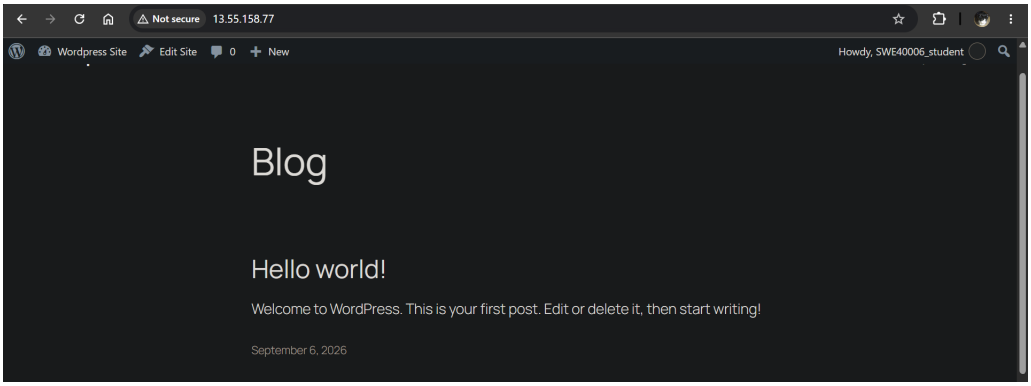

Figure 38: The web application was successfully restored.